# Project 4: Nonlinear Equation Solver – Experiments and Testing Report

## Introduction

Say you need the root of an equation you can't solve by hand. Which numerical method do you reach for, and how do you know it'll work? That's the question behind Project 4. Our team wrote two classic solvers, Bisection and Newton's method, along with a Diagnostics helper that prints iteration tables. As Member 4, my job was to see how they actually behave. I wrote `visualisation.py`, which runs the four required experiments and saves the plots, and `test_visualisation.py`, which makes sure it all runs properly. Below I explain how the code is set up, what each experiment taught us, and how I tested it.

## How the code is organised

Everything sits inside a single `Experiments` class. I didn't rewrite any solver logic. The class simply calls my teammates' `Bisection` and `Newton` classes and borrows the table printer from `Diagnostics`, so everyone's work stays in their own file.

A few choices were made on purpose:

- **Fresh solver per run.** The helpers `run_bisection` and `run_newton` create a new solver every time, so one run's history can never leak into the next.
- **Four test problems in one place.** Each function is stored with its derivative: $x^2 - 2$, $x^3 - x$, $x^3 - 2x + 2$ and $\arctan(x)$.
- **Failures are reported, not hidden.** `print_summary` handles runs with no residual and prints "N/A" instead of crashing.
- **Plots go to a `plots/` folder.** A shared `_finish_plot` method applies the same formatting and saves each figure at 160 dpi.

## The four experiments

**Experiment 1: both methods on $x^2 - 2$.** The positive root is $\sqrt{2}$. I ran Bisection on [1, 2] and Newton from $x_0 = 1$, both to a tolerance of $10^{-10}$, and plotted the true error on a log scale. Bisection gives an almost straight line, because every step just halves the interval, and it takes roughly 33 steps to get there. Newton's curve plunges and is done in a handful of steps, since the number of correct digits roughly doubles each time.

**Experiment 2: does the starting guess matter?** It does. The function $x^3 - x$ has three roots, −1, 0 and 1, and I ran Newton from eight different starting points between −2 and 2. Most of them head for the nearest root, but things get odd around ±0.58, where the derivative is close to zero. Starting at 0.5 throws the iteration all the way over to −1, and starting at 0.6 flings it far to the right before it finally settles on 1. Newton converges, but *which* root you end up with can hinge on a tiny change at the start.

**Experiment 3: when Newton struggles.** Three cases, each with its own failure:

- **Zero derivative.** For $x^2 - 2$ starting at 0, the derivative is zero, so the first Newton step cannot be taken. The solver stops and reports a failure.
- **A cycle.** For $x^3 - 2x + 2$ starting at 0, the iterates bounce between 0 and 1 and never settle. I capped this run at 10 iterations.
- **A bad start.** For arctan(x) starting at 2, each step lands further from the root at 0 instead of closer, so the iteration diverges. Bisection on [−2, 3] solves the same problem without trouble.

**Experiment 4: how tolerance changes the cost.** I tightened the tolerance from $10^{-2}$ to $10^{-12}$ and recorded the iterations, residual and true error for each method. Bisection needs about 3.3 more steps for every extra digit of accuracy, so its count keeps climbing. Newton's barely changes. Getting more precision costs the two methods very differently.

## Testing

The test file uses pytest and has three tests. The first two check that `run_bisection` and `run_newton` both find $\sqrt{2}$ for $x^2 - 2$, that they report convergence, that the residual is small, and that the history length matches the iteration count. The third runs all four experiments from start to finish and checks that the four PNG files exist and aren't empty, and that "All experiments completed." gets printed.

Two small choices make the tests easier to live with. Matplotlib is set to its non-interactive `Agg` mode, so the tests work on a machine without a display. And each test saves into pytest's `tmp_path`, so running them never fills up the real `plots/` folder.

There is a gap, though. The tests prove the experiments run and produce files, but they don't look at what the plots show or check the iteration counts for the tricky cases in Experiment 3. Adding assertions for those would be the next thing to do.

## Conclusion

Neither method is the clear winner. Newton's method is much faster and stays quick even at tight tolerances, but it needs a decent starting point, a derivative that isn't zero, and a bit of luck about where the iteration wanders. Bisection is slower but steady: if the

function changes sign across the interval, it will find the root. A sensible plan in practice is to let bisection get you close, then hand over to Newton to finish. What I'll remember most is how much a plot can show that a final answer can't, especially when a method fails.